Driver Monitoring System in Python using MediaPipe: computation of eye and head gaze positions, EAR and PERCLOS

Belgrano Francesco Saverio (333976), Vadacca Lorenzo (331703)

I. Introduction

This project implements a real-time Driver Monitoring System (**DMS**) using *Python*, *MediaPipe* and *OpenCV*. The system detects driver fatigue and distraction by analyzing facial landmarks and eye closure metrics such as **EAR** (Eye Aspect Ratio) and **PERCLOS** (PERcentage of eye CLOSure). The implementation fulfills the assignment requirements by computing these metrics and triggering alerts when safety thresholds are crossed.

II. Theoretical background

A. Head Pose Estimation

To determine the driver's head orientation, a subset of significant facial landmarks (e.g., eye corners, nose tip) is extracted. These points are used to solve a Perspective-n-Point (PnP) problem, estimating the rotation (pitch, yaw, roll) of the head with respect to the camera. The process involves:

1) Collecting 2D and 3D coordinates of key facial points Defining the camera matrix and distortion parameters.
2) Using `cv2.solvePnP` to estimate the rotation vector.
3) Converting the rotation vector to a rotation matrix with `cv2.Rodrigues`
4) Extracting Euler angles (pitch, yaw, roll) with `cv2.RQDecomp3x3`

B. Eye Gaze Estimation

- **3D gaze**: calculated using the same PnP approach as head pose, but focusing on eye landmarks
- **2D gaze**: If 3D accuracy is insufficient, the position of the iris center relative to the center of the eye can be used to estimate the gaze direction.

C. Eye Aspect Ratio (EAR)

EAR is a geometric measure that is used to estimate eye openness. It is calculated as follows:follows:

$$\text{EAR} = \frac{|y_2 - y_6| + |y_3 - y_5|}{2 \cdot |x_1 - x_4|}$$

- Numerator: sum of vertical distances between eyelid landmarks
- Denominator: horizontal distance between the corners of the eyes
- When the eye closes, the numerator decreases, reducing the EAR

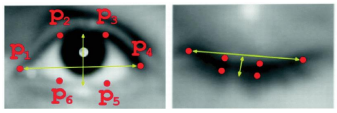


Fig. 1. Eye landmarks to compute EAR

D. PERCLOS

PERCLOS quantifies the percentage of time that the eyes are at least 70% (P70) or 80% (P80) closed over a given interval, a standard metric for detecting drowsiness. It is calculated as follows:

$$\text{PERCLOS} = \frac{t_3 - t_2}{t_4 - t_1}$$

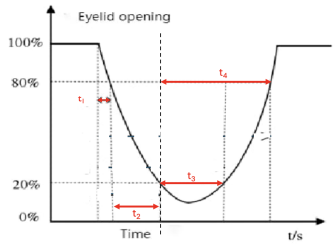


Fig. 2. Eyelid opening

- t_2: Eyes transition from largest to 80% open
- t_2: Eyes transition from 80% to 20% open
- t_3: Eyes transition from 20% closed to 20% open
- t_3: Eyes transition from 20% closed to 80% open

III. Code structure and functionality

A. Importing libraries (lines 1-5)

`cv2`, `mediapipe`, `numpy`, `time` libraries are imported for image processing, facial landmark detection, numerical operations, and timing:

B. MediaPipe FaceMesh setup (lines 7-17)

MediaPipe's FaceMesh model is initialized to detect the face with detailed landmark refinement and specified confidence thresholds. Drawing utilities are also prepared for optional visualization.

C. Video capture initialization (line 20)

The default webcam is opened for real-time video capture.

D. Landmark variables initialization, extraction and assignment (lines (77-160

Variables are prepared to store specific facial landmarks and coordinate arrays for pose estimation and gaze tracking.

For the detected face and its landmarks, the code assigns the corresponding 2D coordinates (scaled by image size) to the appropriate variable, draws circles for visualization, and collects sets of points for pose and gaze estimation.

E. Conversion to NumPy Arrays (lines 162-167)

Lists of landmark coordinates are converted to NumPy arrays for mathematical operations.

```
face_2d = np.array(face_2d, dtype=np.float64)
face_3d = np.array(face_3d, dtype=np.float64)
left_eye_2d = np.array(left_eye_2d,
 ↪  dtype=np.float64)
left_eye_3d = np.array(left_eye_3d,
 ↪  dtype=np.float64)
right_eye_2d = np.array(right_eye_2d,
 ↪  dtype=np.float64)
right_eye_3d = np.array(right_eye_3d,
 ↪  dtype=np.float64)
```

F. Camera matrix and distortion parameters (lines 169-175)

The camera intrinsic matrix is set up and lens distortion is assumed to be zero for pose estimation.

G. Pose estimation and visualization (lines 177-236)

The code estimates the 3D pose (rotation and translation) of the head and eyes using `cv2.solvePnP`, converts rotation vectors to matrices, and extracts Euler angles (pitch, yaw, roll).

The estimated direction of the nose and eyes is projected onto the image, and lines are drawn for visual feedback.

```
nose_3d_projection, jacobian =
 ↪  cv2.projectPoints(nose_3d , rot_vec , trans_vec
 ↪  , cam_matrix , dist_matrix)
cv2.line(image, p1_nose, p2_nose, (255,0,0), 3)

left_eye_3d_projection, jacobian =
 ↪  cv2.projectPoints(left_eye_3d , rot_vec ,
 ↪  trans_vec , cam_matrix , dist_matrix)
x_left_eye_2d, y_left_eye_2d = left_eye_2d[4]
p1_left_eye = (int(x_left_eye_2d),
 ↪  int(y_left_eye_2d))
p2_left_eye = (int(x_left_eye_2d + yaw * 10),
 ↪  int(y_left_eye_2d - pitch * 10))
cv2.line(image, p1_left_eye, p2_left_eye,
 ↪  (255,0,0), 3)

right_eye_3d_projection, jacobian =
 ↪  cv2.projectPoints(right_eye_3d , rot_vec ,
 ↪  trans_vec , cam_matrix , dist_matrix)
x_right_eye_2d, y_right_eye_2d = right_eye_2d[4]
p1_right_eye = (int(x_right_eye_2d),
 ↪  int(y_right_eye_2d))
p2_right_eye = (int(x_right_eye_2d + yaw * 10),
 ↪  int(y_right_eye_2d - pitch * 10))
cv2.line(image, p1_right_eye, p2_right_eye,
 ↪  (255,0,0), 3)
```

The code then computes geometric properties of the eyes and visualizes them, annotating the image with iris and eye center coordinates.

H. Frame rate control (lines 238-247)

To limit processing speed and ensure stability, the frame rate is capped. The actual frame rate is calculated for performance monitoring.

I. Thresholds and parameters (lines 249-254)

Thresholds for drowsiness and distraction detection are defined.

```
HIGH_EAR = 0.8
PERCLOS_ALERT_THRESHOLD = 0.4
DISTRACTION_THRESHOLD = 30
HIGH_EAR_DURATION = 10
```

J. Gaze deviation alert (lines 256-260)

If the head pose angles exceed the distraction threshold, an alert is displayed.

```
if 'pitch' in locals() and 'yaw' in locals() and
 ↪  'roll' in locals():
    combined_deviation = max(abs(pitch), abs(yaw),
     ↪  abs(roll))
    if combined_deviation > DISTRACTION_THRESHOLD:
        cv2.putText(image, "ALERT: Driver
         ↪  Distracted (Gaze Deviation)", (20,
         ↪  250), cv2.FONT_HERSHEY_SIMPLEX, 1 , (0,
         ↪  0, 255), 2)
```

K. Calculation of EAR and drowsiness detection (lines 262-295)

The function computes the Eye Aspect Ratio (EAR) for a given eye using the vertical and horizontal distances between landmarks.

```
def calculate_ear(eye_top, eye_bottom, eye_left,
 ↪  eye_right):
    vertical_1 = abs(eye_top[1] - eye_bottom[1])
    horizontal = abs(eye_left[0] - eye_right[0])
    return vertical_1 / horizontal if horizontal !=
     ↪  0 else 0
```

EAR is calculated for both eyes and their average is used as a measure of eye openness.

```
if point_LEL and point_LER and point_LET and
 ↪  point_LEB:
    left_ear = calculate_ear(point_LET, point_LEB,
     ↪  point_LEL, point_LER)
else:
    left_ear = 0

if point_REL and point_RER and point_RET and
 ↪  point_REB:
    right_ear = calculate_ear(point_RET, point_REB,
     ↪  point_REL, point_RER)
else:
    right_ear = 0

avg_ear = (left_ear + right_ear) / 2
```

The average EAR is rescaled so that a specific value (namely 0.39, estimated empirically) maps to 0.8, and the result is clamped between 0 and 1.

```
normalized_ear = 3.333 * avg_ear - 0.5
normalized_ear = max(0.0, min(1.0, normalized_ear))
```

If the normalized EAR stays above the threshold for a set duration, a drowsiness alert is triggered.

```
if normalized_ear > HIGH_EAR:
    if high_ear_start_time is None:
        high_ear_start_time = current_time
    elif current_time - high_ear_start_time >=
    ↪  HIGH_EAR_DURATION:
        cv2.putText(image, "ALERT: Drowsiness
        ↪  detected (EAR high)", (20, 300),
        ↪  cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0,
        ↪  255), 2)
else:
    high_ear_start_time = None
```

L. Calculation of PERCLOS and sleepiness detection (lines 298-342)

After having implemented an event time logic to be able to reconstruct the right value of PERCLOS, PERCLOS itself is calculated as the ratio of eye-closed time to the total cycle. If it exceeds the threshold, an alert is displayed and state variables are reset.

```
if normalized_ear > 0.8 and not in_closing and not
↪  in_closed and not in_opening and t1_start == 0:
    t1_start = time.time()
    print("Restart cycle")
elif not in_opening and not in_closing and 0.2 <
↪  normalized_ear <= 0.8 and t1_start != 0:
    t1 = time.time() - t1_start
    t1_start = 0  # Reset for the next cycle
    t2_start = time.time()
    in_closing = True
    print("t1 (100% → 80%):", t1)
elif in_closing and normalized_ear <= 0.2:
    t2 = time.time() - t2_start
    t3_start = time.time()
    t4_start = time.time()
    in_closing = False
    in_closed = True
    print("t2 (80% → 20%):", t2)
elif in_closed and normalized_ear >= 0.2:
    t3 = time.time() - t3_start
    in_closed = False
    in_opening = True
    print("t3 (20% → 0 → 20%):", t3)
elif in_opening and normalized_ear > 0.8:
    t4 = time.time() - t4_start
    in_opening = False
    print("t4 (20% → 0 → 80%):", t4)
if (t4 - t1) > 0:
    perclos = (t3 - t2) / (t4 - t1)
    # Reset state for next cycle
    t1 = t2 = t3 = t4 = 0
    t1_start = t2_start = t3_start = t4_start = 0
    in_closing = in_closed = in_opening = False
    if perclos > PERCLOS_ALERT_THRESHOLD:
        cv2.putText(image, "ALERT: Drowsiness
        ↪  Detected (PERCLOS)", (20,
        ↪  350),cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
        ↪  0, 255), 2)
else:
    perclos = 0
```

M. Display EAR and PERCLOS (lines 347-249)

The current EAR and PERCLOS values are displayed on the video frame.

N. Show frame and exit condition (lines 351-355)

The processed frame is displayed in a window, and the loop exits if the ESC key is pressed.

IV. Validation and Conclusions

The developed system demonstrates the effectiveness of driver monitoring based on computer vision. It is an excellent foundation for a real-world DMS, with the possibility of extending it to more complex analysis and intelligent alerts.

The alert notifies the driver when his gaze deviates, namely the angles of the eyes and of the face (nose tip) are above the 30 degrees threshold.

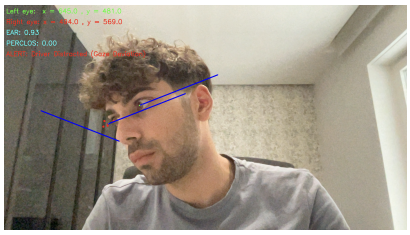


Fig. 3. Gaze deviation alert

The alert notifies the driver in case of drowsiness, namely when the EAR is above 80% for more than 10 seconds.

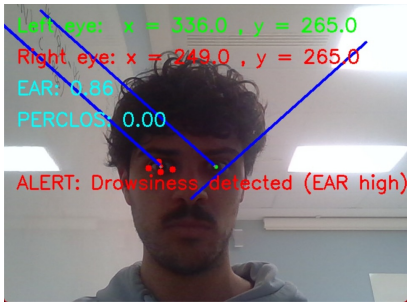


Fig. 4. High EAR alert

The alert notifies the driver in case of sleepiness, namely when the PERCLOS is above a supposed threshold.

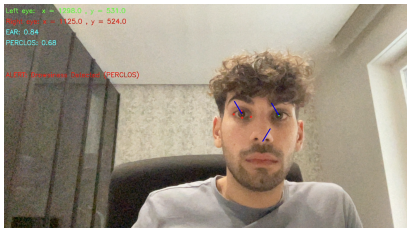


Fig. 5. High PERCLOS alert